

FPLN: PRÁCTICA 3

Manual de usuario

El proyecto ha sido desarrollado en Python 3 utilizando principalmente las librerías TensorFlow y NumPy. El objetivo de la práctica es implementar un parser de dependencias basado en transiciones utilizando el algoritmo Arc-Eager junto con un modelo neuronal encargado de predecir las acciones del parser.

El sistema trabaja con archivos en formato CoNLL-U y utiliza el corpus *en_partut-ud* dividido en entrenamiento, desarrollo y test.

Para ejecutar el sistema es suficiente con lanzar el archivo *main.py*. Toda la ejecución del pipeline se realiza automáticamente desde este archivo, incluyendo la carga de datos, el entrenamiento, la evaluación y la generación de árboles sintácticos.

Durante la primera parte de la ejecución se cargan los árboles sintácticos del corpus y se eliminan las oraciones no proyectivas del conjunto de entrenamiento y desarrollo.

Una vez preparados los datos, el sistema genera automáticamente las muestras de entrenamiento utilizando el oráculo del parser. Estas muestras representan estados del parser junto con la transición correcta que debe aplicarse en cada caso. Posteriormente, dichas muestras se utilizan para entrenar el modelo neuronal.

El modelo implementado es un perceptrón multicapa sencillo construido con TensorFlow/Keras. La red neuronal utiliza de entrada embeddings (tanto de palabras como de etiquetas POS) y predice simultáneamente tanto la acción de parsing como la etiqueta de dependencia correspondiente. Se optó por una arquitectura relativamente simple (tan solo una capa oculta) porque el objetivo principal de la práctica era comprender el funcionamiento general de un parser basado en transiciones y no maximizar el rendimiento mediante arquitecturas complejas.

Tras el entrenamiento, el modelo procesa el conjunto de test y genera automáticamente árboles sintácticos para cada oración. Los resultados se guardan en formato CoNLL-U dentro del archivo *test_output.conllu*. Posteriormente se aplica un postprocesado para corregir posibles errores estructurales y generar una versión final más consistente del resultado.

Una vez generados los árboles de test, el sistema evalúa automáticamente el modelo utilizando distintas métricas estándar de Universal Dependencies, entre ellas UAS, LAS, MLAS y BLEX. Además de los valores finales, también se muestran métricas detalladas de precisión, recall y F1 para diferentes aspectos del análisis sintáctico.

Decisiones de diseño

Se decidió utilizar embeddings POS de tamaño 32 porque las etiquetas gramaticales contienen mucha menos variabilidad y complejidad semántica que las palabras. Mientras que el vocabulario léxico puede ser muy amplio y requerir embeddings más grandes para capturar información contextual, el conjunto de etiquetas POS es reducido y relativamente simple. Por ello, un tamaño de 32 dimensiones resulta suficiente para representar adecuadamente la información morfosintáctica sin incrementar innecesariamente el número de parámetros del modelo ni el coste computacional.

La función de pérdida utilizada fue *sparse_categorical_crossentropy* debido a que las salidas esperadas del modelo se representan mediante identificadores enteros y no mediante vectores one-hot encoding. Cada acción del parser y cada etiqueta de dependencia se codifican directamente como un índice numérico. Esta función de pérdida es la opción más adecuada en este caso porque TensorFlow puede trabajar directamente con clases enteras, evitando conversiones adicionales y reduciendo tanto el consumo de memoria como el tiempo de entrenamiento.

El entrenamiento se limitó a 5 epochs porque se observó que aumentar demasiado el número de iteraciones provocaba sobreajuste. A partir de cierto punto, el modelo seguía mejorando sobre los datos de entrenamiento pero empeoraba en desarrollo y test, lo que indica pérdida de capacidad de generalización. Utilizar únicamente 5 epochs permitió mantener un equilibrio razonable entre aprendizaje y generalización, evitando que el modelo memorizase excesivamente las muestras de entrenamiento.

Otro aspecto importante fue el tratamiento de dependencias desconocidas. Durante algunas pruebas el modelo tendía a producir etiquetas vacías o inválidas. Para evitar árboles mal formados se decidió utilizar la etiqueta genérica *"dep"* como fallback cuando fuera necesario.

Discusión de resultados

```
Evaluation results:
Total loss: 1.1388
Action accuracy: 0.8511
Dependency accuracy: 0.8419

===== EVALUATION =====
Tokens      | Precision: 100.00 | Recall: 100.00 | F1: 100.00
Sentences   | Precision: 100.00 | Recall: 100.00 | F1: 100.00
Words       | Precision: 100.00 | Recall: 100.00 | F1: 100.00
UPOS        | Precision: 100.00 | Recall: 100.00 | F1: 100.00
XPOS        | Precision: 100.00 | Recall: 100.00 | F1: 100.00
UFeats      | Precision: 100.00 | Recall: 100.00 | F1: 100.00
AllTags     | Precision: 100.00 | Recall: 100.00 | F1: 100.00
Lemmas      | Precision: 100.00 | Recall: 100.00 | F1: 100.00
UAS         | Precision: 75.50  | Recall: 75.50  | F1: 75.50
LAS         | Precision: 68.40  | Recall: 68.40  | F1: 68.40
CLAS        | Precision: 54.07  | Recall: 55.59  | F1: 54.82
MLAS        | Precision: 52.87  | Recall: 54.35  | F1: 53.60
BLEX        | Precision: 54.07  | Recall: 55.59  | F1: 54.82

LAS F1 Score: 68.40
MLAS Score: 53.60
BLEX Score: 54.82
```

Los resultados obtenidos muestran un comportamiento coherente para un parser basado en transiciones utilizando un MLP relativamente sencillo. En primer lugar, las métricas de entrenamiento indican que el modelo aprende correctamente la tarea principal. La precisión en la predicción de acciones alcanza un 85.11%, mientras que la precisión en las etiquetas de dependencia llega al 84.19%. Esto indica que el clasificador es capaz de aprender con bastante fiabilidad tanto las transiciones del algoritmo Arc-Eager como las relaciones sintácticas asociadas.

Las métricas estructurales básicas aparecen con valores del 100% porque el sistema trabaja sobre árboles ya tokenizados y segmentados correctamente. Por tanto, Tokens, Sentences, Words, UPOS, XPOS, UFeats, AllTags y Lemmas no representan realmente la dificultad principal del problema, ya que el parser no modifica esa información durante la inferencia.

Las métricas más relevantes son UAS y LAS. El modelo obtiene un UAS de 75.50%, lo que significa que aproximadamente tres de cada cuatro palabras reciben correctamente su cabeza sintáctica. Este resultado puede considerarse razonable para un parser basado en MLP sin mecanismos contextuales avanzados.

El LAS desciende hasta 68.40%, lo cual era esperable. Mientras que UAS únicamente evalúa si la dependencia apunta al nodo correcto, LAS exige además que la etiqueta sintáctica también sea correcta. La diferencia entre ambas métricas refleja que parte de los errores del sistema no están en la estructura del árbol, sino en la clasificación concreta de las relaciones gramaticales.

Las métricas CLAS, MLAS y BLEX son más estrictas y por ello presentan valores inferiores. En especial MLAS, que incorpora simultáneamente información morfológica y dependencias

funcionales, resulta significativamente más complejo. El valor de 53.60% muestra que el modelo todavía tiene dificultades cuando necesita capturar información sintáctica más rica y relaciones de largo alcance.

En general, los resultados son consistentes con las limitaciones de la arquitectura utilizada. El uso de un MLP permite obtener un parser funcional y relativamente eficiente, pero carece de mecanismos avanzados para modelar contexto secuencial complejo.

Aun así, el sistema implementado cumple correctamente los objetivos de la práctica: generar árboles sintácticos válidos, aprender las transiciones del algoritmo Arc-Eager y alcanzar resultados razonables sobre el conjunto de test utilizando únicamente un parser de transición clásico y una red neuronal sencilla.